

Deep Learning

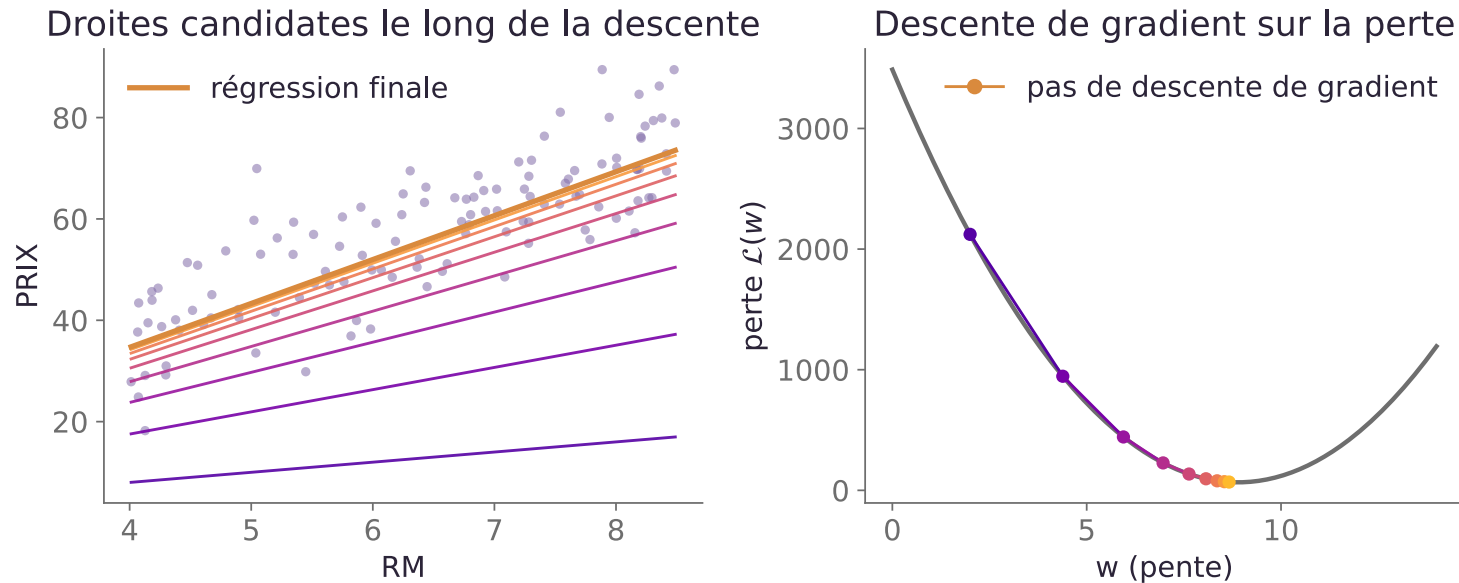
2. Descente de Gradient Stochastique

Romain Tavenard

Optimisation en général

- **Objectif** : ajuster les paramètres du modèle afin de minimiser l'erreur
- Mesurer l'erreur
 - Via une fonction de coût / fonction de perte
 - Exemple typique : l'erreur quadratique moyenne en régression

Descente de gradient



1. Choisir une fonction de perte (différentiable) à minimiser :

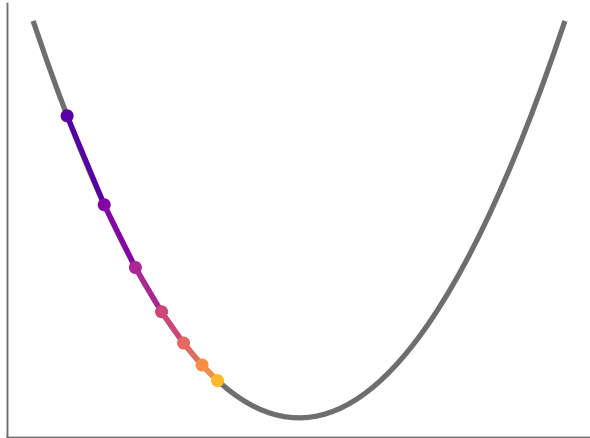
$$\begin{aligned}\mathcal{L}(w, \{x_i, y_i\}) &= \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(w, x_i, y_i) \\ &= \frac{1}{n} \sum_{i=1}^n (\varphi(w^t x_i) - y_i)^2\end{aligned}$$

2. Appliquer itérativement des pas de descente de gradient

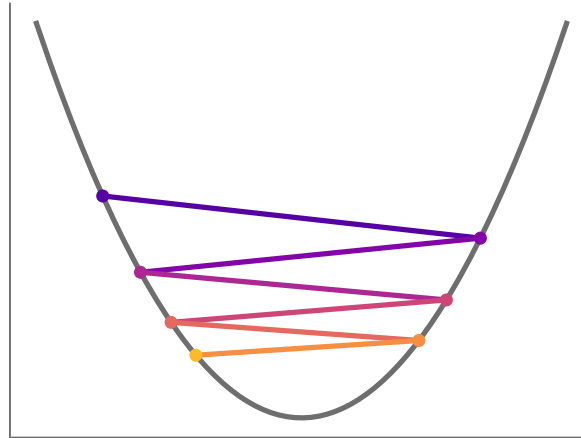
$$w^{(t+1)} \leftarrow w^{(t)} - \rho \nabla_w \mathcal{L}(w^{(t)})$$

La descente de gradient en pratique

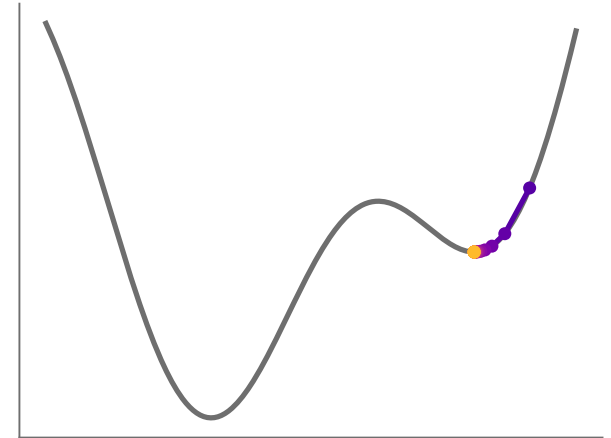
Taux d'apprentissage trop petit
(lent)



Taux d'apprentissage trop grand
(oscille)



Minimum local
/ plateau



Pièges du taux d'apprentissage et minima locaux

Descente de Gradient Stochastique (SGD)

Algorithme 1 : Descente de gradient

Pour chaque *epoch* :

1. Calculer le gradient sur le **jeu de données complet** :

$$\begin{aligned} G &= \nabla_w \mathcal{L}(w^{(t)}) \\ &= \frac{1}{n} \sum_{i=1}^n \nabla_w \mathcal{L}_i(w^{(t)}) \end{aligned}$$

2. Mettre à jour w :

$$w^{(t+1)} \leftarrow w^{(t)} - \rho G$$

Descente de Gradient Stochastique (SGD)

Algorithme 1 : Descente de gradient

Pour chaque *epoch* :

1. Calculer le gradient sur le **jeu de données complet** :

$$\begin{aligned} G &= \nabla_w \mathcal{L}(w^{(t)}) \\ &= \frac{1}{n} \sum_{i=1}^n \nabla_w \mathcal{L}_i(w^{(t)}) \end{aligned}$$

2. Mettre à jour w :

$$w^{(t+1)} \leftarrow w^{(t)} - \rho G$$

Algorithme 2 : SGD par *mini-batches*

Pour chaque *epoch* :

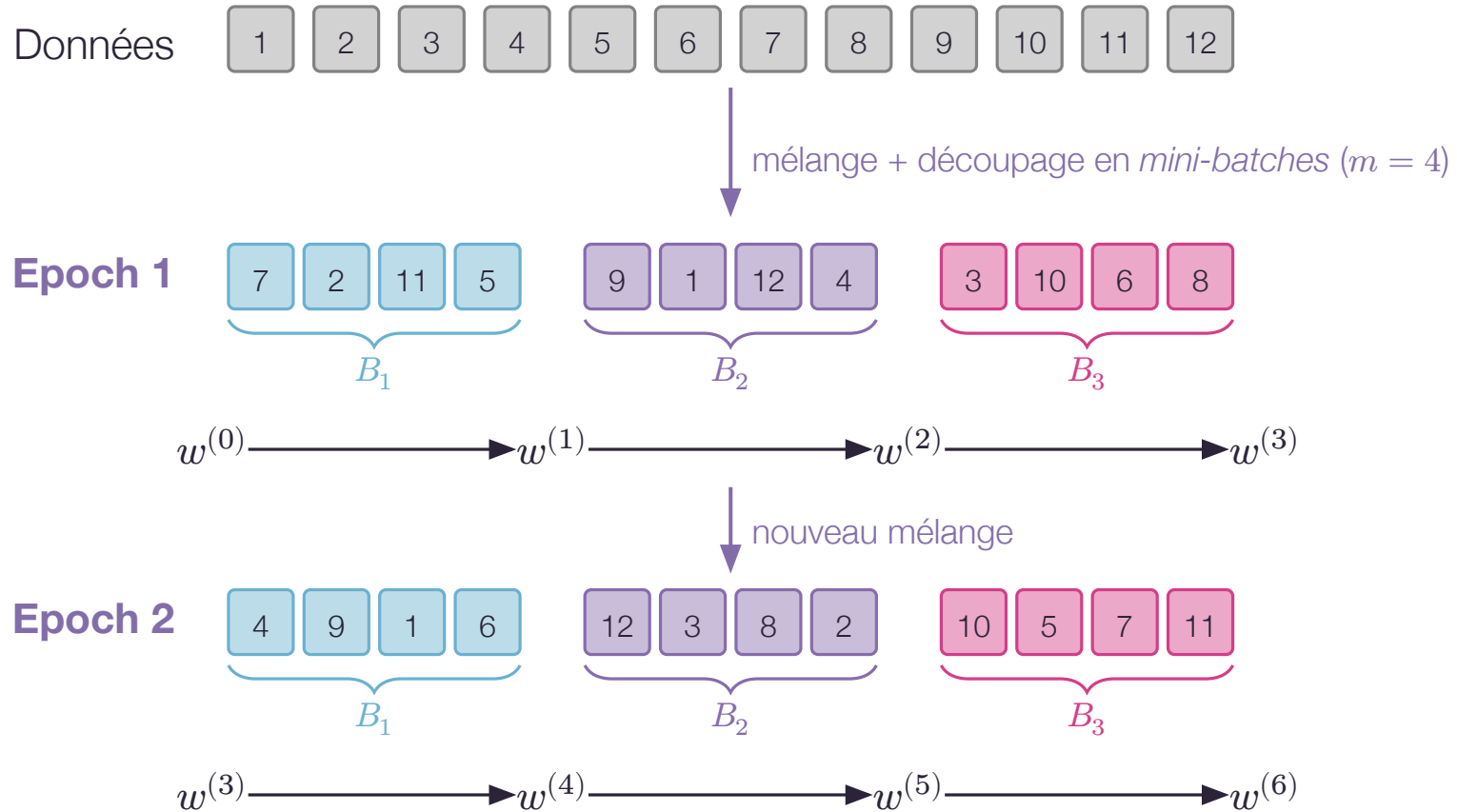
1. Découper les données en *mini-batches* de taille m
2. Pour tout *mini-batch* B :
 1. Estimer le gradient sur le ***mini-batch*** :

$$G \approx \frac{1}{m} \sum_{i \in B} \nabla_w \mathcal{L}_i(w^{(t)})$$

2. Mettre à jour w :

$$w^{(t+1)} \leftarrow w^{(t)} - \rho G$$

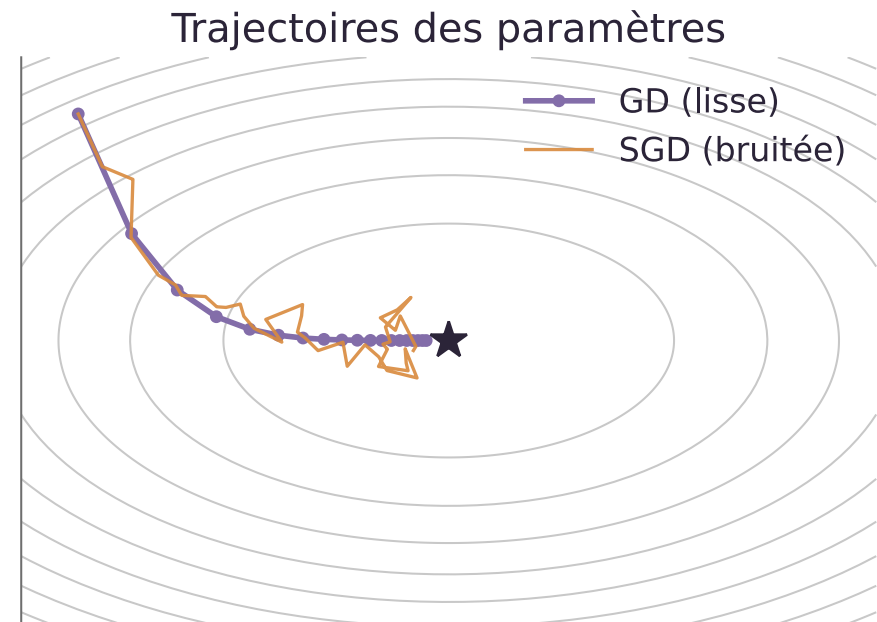
Mini-batches et epochs



Epoch : un passage complet sur les données, soit n/m mises à jour

Descente de gradient contre descente de gradient stochastique

- Inconvénients du SGD
 - Sujet à une forte variance
- Avantages du SGD
 - Mise à jour des poids plus rapide (à chaque *mini-batch*)
 - Permet d'échapper aux minima locaux dans les cas non convexes



GD : trajectoire lisse. SGD : trajectoire bruitée, même minimum

Variantes du SGD : zoom sur Adam (1/2)

- **Idée 1**
 - SGD classique : petits pas sur les pentes douces, immenses sur les pentes raides
 - objectif : que chaque paramètre progresse à un **rythme comparable**
- **Comment :** diviser chaque gradient par **sa propre magnitude** → la taille s'annule, seul le **signe** subsiste :

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \rho \frac{\nabla_{\mathbf{w}} \mathcal{L}}{\underbrace{\sqrt{\nabla_{\mathbf{w}} \mathcal{L} \odot \nabla_{\mathbf{w}} \mathcal{L}}}_{\text{signe}(\nabla_{\mathbf{w}} \mathcal{L})}}$$

→ un **pas de même taille** dans toutes les directions

Variantes du SGD : zoom sur Adam (2/2)

- **Idée 2** : ajouter du *momentum*
 - le gradient d'un seul *mini-batch* est bruité
 - conserver une **moyenne glissante** → comme une balle qui dévale une pente [distill]
- **Comment** : lisser le gradient (et son carré) en moyennes $\mathbf{m}$, $\mathbf{s}$:

$$\mathbf{m}^{(t+1)} = \beta_1 \mathbf{m}^{(t)} + (1 - \beta_1) \nabla_w \mathcal{L}$$

$$\mathbf{s}^{(t+1)} = \beta_2 \mathbf{s}^{(t)} + (1 - \beta_2) \nabla_w \mathcal{L} \odot \nabla_w \mathcal{L}$$

- **Mise à jour complète** utilisant les quantités lissées :

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \rho \hat{\mathbf{m}} \oslash \sqrt{\hat{\mathbf{s}} + \varepsilon}$$

Conclusion

- Descente de Gradient Stochastique
 - Estimations du gradient calculées sur des *mini-batches*
 - Mises à jour plus fréquentes
 - Adam est une variante extrêmement performante